



Smart Book System

Technical Guide for Beginners

Node.js

Express

MongoDB

JWT Auth

Version 1.0 | Beginner-Friendly Edition

Table of Contents

1. [System Overview](#)
2. [Architecture & How Requests Flow](#)
3. [Understanding Middleware \(protect & adminOnly\)](#)
4. [The Three Database Collections](#)
5. [Every Feature — Step by Step](#)
6. [The Borrow Lifecycle \(State Machine\)](#)
7. [Quick Reference Summary Table](#)
8. [Glossary of Terms](#)

1. System Overview

The **Smart Book System** is a library management web application. It lets users browse books, express interest, request to borrow, and read books — while admins manage books, users, and borrow records.

What this guide covers

This guide explains, in plain language, **what happens behind the scenes** when a user or admin takes an action. For every feature you will see:

-  **Who** can perform the action (Public / Logged-in User / Admin)
-  **Which middleware** runs (e.g. `protect` , `adminOnly`)
-  **What database action** is triggered (e.g. create, find, update, delete)

KEY IDEA

Think of the system as a **restaurant**: the *route* is the menu item, the *middleware* is the security guard checking your ID, and the *controller* is the chef who actually prepares the dish (talks to the database).

Two Types of Users

Role	What they can do
USER	Register, login, browse books, search, express interest, request checkout, view own records, read borrowed books
ADMIN	Everything a user can do, plus : create/update/delete books, manage all users, view all borrow records, update any record status

2. Architecture & How Requests Flow

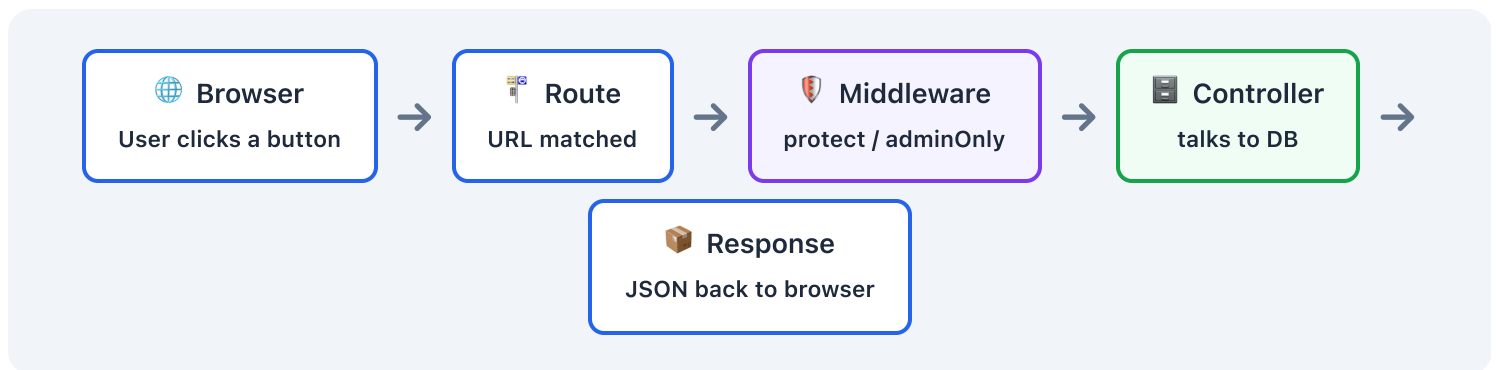
High-Level Architecture

The Layers

The app is built in layers. Each layer has one job:

1. **Frontend (public/ folder)** — HTML + JavaScript pages the user sees in the browser.
2. **Routes (routes/ folder)** — Define the URL paths and decide which middleware + controller to call.
3. **Middleware (middleware/ folder)** — Security checks that run *before* the controller.
4. **Controllers (controllers/ folder)** — The "brain" that talks to the database and sends back a response.
5. **Models (models/ folder)** — Define the shape of data stored in MongoDB.

How a Request Flows Through the System



WHAT HAPPENS AT EACH STEP

1. **Browser** sends an HTTP request (e.g. `GET /api/books`).
2. **Route** matches the URL and calls the middleware chain.
3. **Middleware** checks authentication (is there a valid token?) and authorization (is this user allowed?). If a check fails, it stops the request and returns an error.
4. **Controller** performs the actual database operation (find, create, update, delete).

5. **Response** is sent back as JSON, which the frontend displays.

Where the Routes Live

URL Prefix	Route File	Purpose
/api/auth	routes/authRoutes.js	Register, login, get my profile
/api/books	routes/bookRoutes.js	Browse, search, read, manage books
/api/borrow	routes/borrowRoutes.js	Interest, checkout, status, records
/api/users	routes/userRoutes.js	Admin user management

3. Understanding Middleware

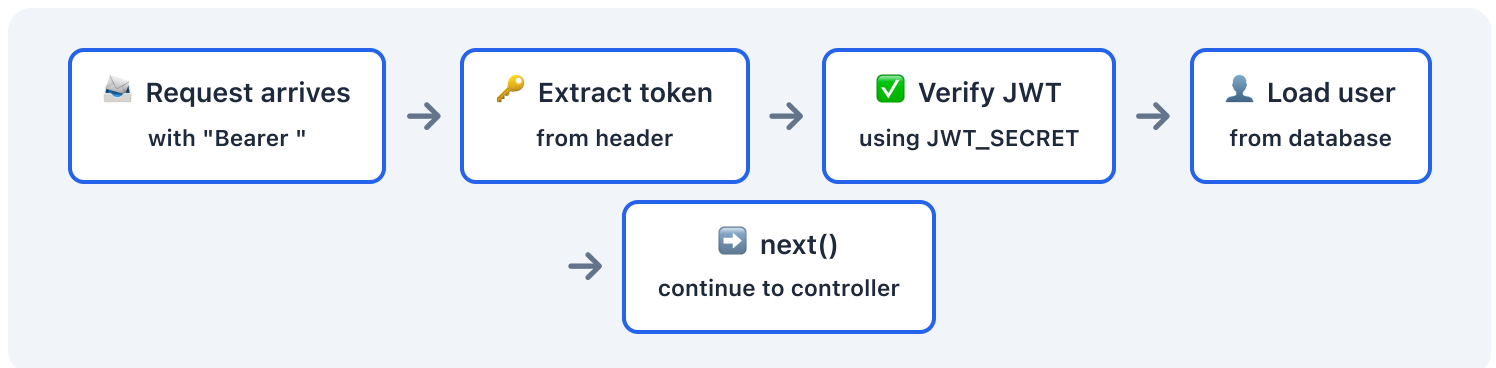
Middleware are functions that run **between** receiving a request and sending a response. They act like **security checkpoints**.

The `protect` Middleware

What it does

The `protect` middleware verifies that the request comes from a **logged-in user**. It does this by checking the **JWT (JSON Web Token)** sent in the request header.

Step-by-step: What `protect` does



🚫 WHEN PROTECT FAILS

- No token** → returns `401 "Not authorized, no token"`
- Invalid/expired token** → returns `401 "Not authorized, token failed"`
- User no longer exists** → returns `401 "User not found"`

The actual code

```
const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    try {
      token = req.headers.authorization.split(' ')[1];
    }
  }
}
```

```

const decoded = jwt.verify(token, process.env.JWT_SECRET);
req.user = await User.findById(decoded.id).select('-password');
if (!req.user) return res.status(401).json({ message: 'User not found' });
next();
} catch (error) {
  return res.status(401).json({ message: 'Not authorized, token failed' });
}
}
if (!token) return res.status(401).json({ message: 'Not authorized, no token' });
};

```

✅ WHAT PROTECT GIVES YOU

After `protect` runs successfully, the controller can access the logged-in user via `req.user` (e.g. `req.user._id`, `req.user.role`).

🔑 The `adminOnly` Middleware

What it does

The `adminOnly` middleware checks that the logged-in user has the **admin** role. It must always be used **after** `protect` (because it needs `req.user`).

```

const adminOnly = (req, res, next) => {
  if (req.user && req.user.role === 'admin') {
    next();
  } else {
    return res.status(403).json({ message: 'Access denied: Admin only' });
  }
};

```

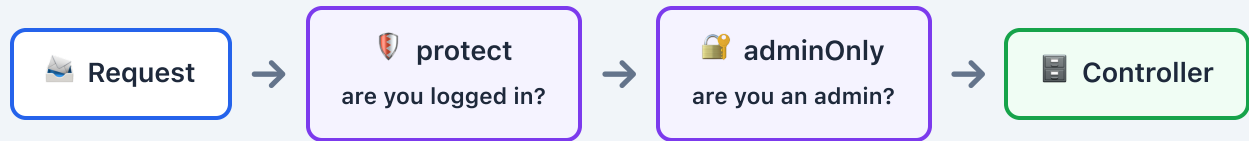
⚠️ IMPORTANT

`adminOnly` returns `403 "Access denied: Admin only"` if the user is not an admin. This is different from `401` (not logged in) — `403` means "you are logged in, but you don't have permission".

How Middleware is Combined

Middleware can be chained. For example, an admin-only route uses both:

```
router.post('/', protect, adminOnly, createBook);
```



4. The Three Database Collections

The app uses **MongoDB** (a NoSQL database). Data is stored in **collections** (like tables in a traditional database). There are three collections, defined by the **models**.

Collection 1: `users` (Model: `User.js`)

Stores information about every user

Field	Type	Description
<code>name</code>	String	User's display name
<code>email</code>	String (unique)	Login email
<code>password</code>	String (hashed)	Password is hashed (never stored in plain text)
<code>role</code>	String	<code>'user'</code> or <code>'admin'</code>
<code>createdAt</code> / <code>updatedAt</code>	Date	Auto-added timestamps

PASSWORD SECURITY

When a user is created, a **pre-save hook** automatically hashes the password using `bcrypt`. The plain password is never stored. When logging in, the system compares the entered password against the stored hash.

Collection 2: `books` (Model: `Book.js`)

Stores information about each book

Field	Type	Description
-------	------	-------------

<code>title</code>	String	Book title
<code>author</code>	String	Book author
<code>isbn</code>	String (unique)	Unique ISBN identifier
<code>totalQty</code>	Number	Total copies the library owns
<code>stock</code>	Number	Currently available copies (defaults to totalQty)
<code>content</code>	String	The book's text content (for reading)

💡 TOTALQTY VS STOCK

totalQty = how many copies the library owns. **stock** = how many are currently available. When someone borrows a book, `stock` decreases. When they return it, `stock` increases.

🔄 Collection 3: `borrowrecords` (Model: BorrowRecord.js)

Tracks the borrowing lifecycle

Field	Type	Description
<code>userId</code>	ObjectId (ref: User)	Which user owns this record
<code>bookId</code>	ObjectId (ref: Book)	Which book this record is about
<code>status</code>	String	<code>interested</code> → <code>pending</code> → <code>borrowed</code> → <code>returned</code>
<code>borrowDate</code>	Date	When the book was actually borrowed
<code>returnDate</code>	Date	When the book was returned

5. Every Feature — Step by Step

Below is every feature in the system. For each one, you'll see the **route**, **who can use it**, **which middleware runs**, and **what database action happens**.

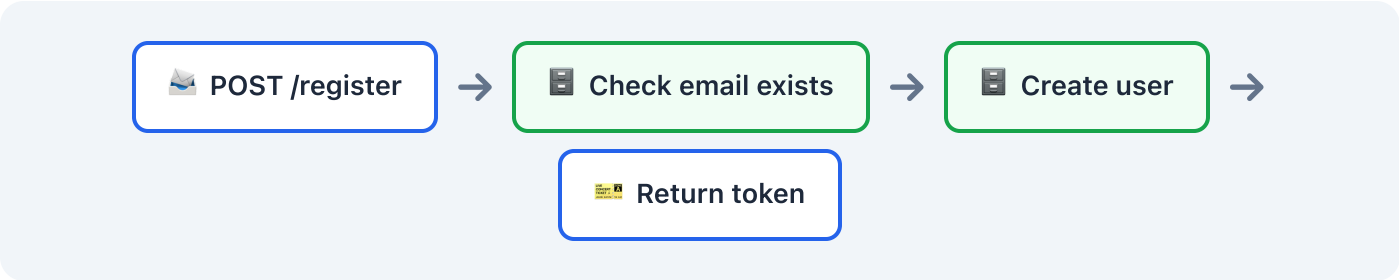
Authentication Features

5.1 Register a New User

PUBLIC

NO MIDDLEWARE

Item	Detail
Route	POST /api/auth/register
Who	Anyone (public)
Middleware	None — this is how you get a token
Database action	User.findOne({ email }) to check duplicates, then User.create(...) to create the user (password auto-hashed)
Response	Returns a JWT token + user info



5.2 Login

PUBLIC

NO MIDDLEWARE

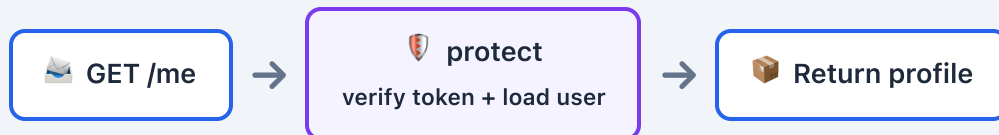
Item	Detail
Route	POST /api/auth/login
Who	Anyone (public)
Middleware	None
Database action	User.findOne({ email }).select('+password') to fetch user with password, then user.matchPassword(password) to verify
Response	Returns a JWT token + user info

5.3 Get My Profile

PRIVATE

protect

Item	Detail
Route	GET /api/auth/me
Who	Any logged-in user
Middleware	protect — verifies the token and loads req.user
Database action	None in the controller — req.user was already loaded by protect (which did User.findById)
Response	Returns the logged-in user's profile



5.4 Get All Books

PUBLIC

NO MIDDLEWARE

Item	Detail
Route	GET /api/books
Who	Anyone (public)
Middleware	None
Database action	<code>Book.find().sort({ createdAt: -1 })</code> — get all books, newest first

5.5 Search Books

PUBLIC

NO MIDDLEWARE

Item	Detail
Route	GET /api/books/search?q=keyword
Who	Anyone (public)
Middleware	None
Database action	<code>Book.find({ \$or: [{ title: regex }, { author: regex }] })</code> — case-insensitive search on title or author

5.6 Get a Single Book

PUBLIC

NO MIDDLEWARE

Item	Detail
Route	GET /api/books/:id

Who Anyone (public)

Middleware None

Database action `Book.findById(req.params.id)` — find one book by its ID

5.7 Read Book Content

PRIVATE

protect

Item	Detail
------	--------

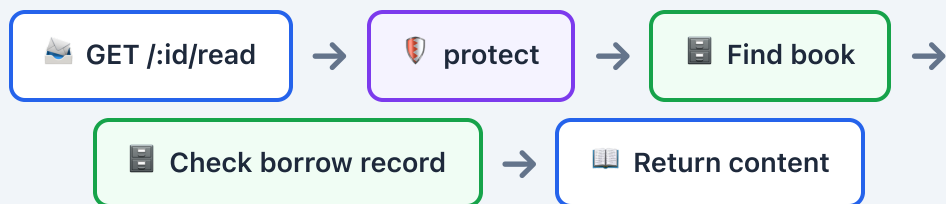
Route	<code>GET /api/books/:id/read</code>
-------	--------------------------------------

Who Logged-in user who has **borrowed** the book, OR any admin

Middleware `protect` — must be logged in

Database action `Book.findById` to get the book, then `BorrowRecord.findOne({ userId, bookId, status: 'borrowed' })` to check the user has an active borrow

Access rule Admin → always allowed. User → only if they have a `borrowed` record for this book



5.8 Create a Book (Admin)

ADMIN

protect + adminOnly

Item	Detail
------	--------

Route	POST /api/books
Who	Admin only
Middleware	protect (logged in) → adminOnly (is admin)
Database action	Book.findOne({ isbn }) to check for duplicate ISBN, then Book.create(...) to create the book



5.9 Update a Book (Admin)

ADMIN

protect + adminOnly

Item	Detail
Route	PUT /api/books/:id
Who	Admin only
Middleware	protect → adminOnly
Database action	Book.findById to load, checks ISBN uniqueness, updates fields, then book.save(). If totalQty changes, stock is adjusted by the difference

5.10 Delete a Book (Admin)

ADMIN

protect + adminOnly

Item	Detail
------	--------

Route	DELETE /api/books/:id
Who	Admin only
Middleware	protect → adminOnly
Database action	Book.findById to load, then book.deleteOne() to remove it

Borrow Features

IMPORTANT

All borrow routes use `router.use(protect)` at the top, which means **every** borrow route requires the user to be logged in. The `protect` middleware runs automatically for all of them.

5.11 Express Interest in a Book

PRIVATE protect

Item	Detail
Route	POST /api/borrow/interest
Who	Any logged-in user
Middleware	protect (via <code>router.use</code>)
Database action	<code>Book.findById</code> to verify book exists, <code>BorrowRecord.findOne</code> to check for existing active record, then <code>BorrowRecord.create({ status: 'interested' })</code>
Result	Creates a record with status <code>interested</code> (like a wishlist)

5.12 Request Checkout (Borrow)

PRIVATE protect

Item	Detail
Route	POST /api/borrow/checkout
Who	Any logged-in user
Middleware	protect
Database action	Book.findById to verify book + check <code>stock > 0</code> , then either <code>record.save()</code> (upgrade existing interested/pending record to <code>pending</code>) or <code>BorrowRecord.create({ status: 'pending' })</code>
Result	Creates/upgrades a record to status <code>pending</code> (request submitted)

5.13 Update Borrow Status

PRIVATE

protect

ADMIN OR OWNER

Item	Detail
Route	PUT /api/borrow/status/:id
Who	Admin (any record) OR the record owner (their own record)
Middleware	protect — then the controller checks ownership
Database action	BorrowRecord.findById to load, checks permission (admin or owner), validates the status transition, then Book.findById to adjust <code>stock</code> , then <code>book.save()</code> + <code>record.save()</code>
Stock effect	→ <code>borrowed : stock -= 1</code> and sets <code>borrowDate</code> . → <code>returned : stock += 1</code> and sets <code>returnDate</code>



 Update stock + record

5.14 Get My Borrow Records

PRIVATE

protect

Item	Detail
Route	<code>GET /api/borrow/my-records</code>
Who	Any logged-in user
Middleware	<code>protect</code>
Database action	<code>BorrowRecord.find({ userId: req.user._id }).populate('bookId', ...)</code> — find the user's records and fill in book details

5.15 Get All Borrow Records (Admin)

ADMIN

protect + adminOnly

Item	Detail
Route	<code>GET /api/borrow/all</code>
Who	Admin only
Middleware	<code>protect</code> (via <code>router.use</code>) → <code>adminOnly</code>
Database action	<code>BorrowRecord.find().populate('userId').populate('bookId')</code> — get all records with user and book details

User Management (Admin)

💡 IMPORTANT

All user management routes use `router.use(protect, adminOnly)` at the top, so **every** route requires the user to be both logged in **and** an admin.

5.16 Get All Users (Admin)

ADMIN

protect + adminOnly

Item	Detail
Route	GET /api/users
Who	Admin only
Middleware	protect → adminOnly
Database action	User.find().select('-password') — get all users (without passwords)

5.17 Create a User (Admin)

ADMIN

protect + adminOnly

Item	Detail
Route	POST /api/users
Who	Admin only
Middleware	protect → adminOnly
Database action	User.findOne({ email }) to check duplicates, then User.create(...) (password auto-hashed). Admin can set the role to admin or user

5.18 Delete a User (Admin)

ADMIN**protect + adminOnly**

Item	Detail
Route	<code>DELETE /api/users/:id</code>
Who	Admin only
Middleware	<code>protect</code> → <code>adminOnly</code>
Database action	<code>User.findById</code> to load, prevents deleting self, then <code>BorrowRecord.deleteMany({</code> <code>userId })</code> to remove the user's borrow records, then <code>user.deleteOne()</code>

6. The Borrow Lifecycle (State Machine)

A borrow record moves through **four states**. Each state represents a stage in the borrowing process. Not every transition is allowed — the system enforces a strict order.

The State Machine Diagram



Allowed Transitions

From	Can go to	What happens
interested	pending	User requests checkout
pending	borrowed or returned	Borrowing: <code>stock -= 1</code> , sets <code>borrowDate</code> . Returning: <code>stock += 1</code> , sets <code>returnDate</code>
borrowed	returned	Returning: <code>stock += 1</code> , sets <code>returnDate</code>
returned	(none)	Final state — cannot change

⚠ INVALID TRANSITIONS ARE REJECTED

For example, you cannot go from `interested` directly to `borrowed` , or from `returned` back to `pending` . The system returns a `400` error: *"Invalid transition from 'X' to 'Y'".*

How Stock Changes Through the Lifecycle

Stock tracking

- **interested / pending** → stock is **unchanged** (no book taken yet)
- **borrowed** → `stock -= 1` (one copy is now out)
- **returned** → `stock += 1` (the copy comes back)

Example: A book has `totalQty = 3` and `stock = 3`. When someone borrows it, `stock` becomes `2`. When they return it, `stock` goes back to `3`.

7. Quick Reference Summary Table

Here is every route in one table, showing who can access it and what middleware runs.

Auth Routes

Method & Path	Access	Middleware	DB Action
POST /api/auth/register	Public	—	Create user
POST /api/auth/login	Public	—	Find + verify user
GET /api/auth/me	Private	protect	None (uses req.user)

Book Routes

Method & Path	Access	Middleware	DB Action
GET /api/books	Public	—	Find all books
GET /api/books/search	Public	—	Find by title/author
GET /api/books/:id	Public	—	Find one book
GET /api/books/:id/read	Private	protect	Find book + check borrow
POST /api/books	Admin	protect + adminOnly	Create book
PUT /api/books/:id	Admin	protect + adminOnly	Update book
DELETE /api/books/:id	Admin	protect + adminOnly	Delete book

Borrow Routes (all require protect)

Method & Path	Access	Middleware	DB Action
---------------	--------	------------	-----------

POST /api/borrow/interest	Private	protect	Create interested record
POST /api/borrow/checkout	Private	protect	Create/upgrade to pending
PUT /api/borrow/status/:id	Private (admin or owner)	protect	Update status + stock
GET /api/borrow/my-records	Private	protect	Find user's records
GET /api/borrow/all	Admin	protect + adminOnly	Find all records

User Management Routes (all require **protect** + **adminOnly**)

Method & Path	Access	Middleware	DB Action
GET /api/users	Admin	protect + adminOnly	Find all users
POST /api/users	Admin	protect + adminOnly	Create user
DELETE /api/users/:id	Admin	protect + adminOnly	Delete user + records

8. Glossary of Terms

Term	Meaning
API	Application Programming Interface — a way for software to talk to each other over HTTP
Route	A URL path (e.g. <code>/api/books</code>) that maps to a handler
Middleware	A function that runs between the request and the response, often for security checks
Controller	The function that handles the request and talks to the database
Model	Defines the structure of data stored in MongoDB
JWT (JSON Web Token)	A secure token that proves a user is logged in. Sent in the <code>Authorization</code> header as <code>Bearer</code>
protect	Middleware that verifies the JWT and loads the logged-in user into <code>req.user</code>
adminOnly	Middleware that checks the user has the <code>admin</code> role
req.user	The logged-in user object, attached by the <code>protect</code> middleware
MongoDB	A NoSQL database that stores data in flexible documents
Collection	A group of documents in MongoDB (like a table)
Document	A single record in a collection (like a row)
Schema	The structure/definition of a document in a collection
bcrypt	A library that hashes passwords so they are never stored in plain text
populate	A Mongoose feature that fills in referenced data (e.g. show book details in a borrow record)
HTTP Status 401	Unauthorized — you are not logged in or your token is invalid
HTTP Status 403	Forbidden — you are logged in but don't have permission

HTTP Status 404

Not Found — the resource doesn't exist

HTTP Status 400

Bad Request — the request is invalid (e.g. invalid status transition)

 YOU'RE ALL SET!

You now understand how the Smart Book System works behind the scenes — from routes and middleware to controllers and database actions. Happy coding!